

OpenC910 IEW 对齐修改总结

概览

对 gem5 O3 CPU 的 IEW (Issue/Execute/Writeback) 阶段进行 OpenC910 架构对齐，涵盖 7 个 Phase 的修改，涉及 21 个文件、约 1123 行变更。

Phase 1：流水线宽度调整

将默认流水线宽度从 gem5 默认的 8 调整为更贴近 C910 实际能力。

src/cpu/o3/BaseO3CPU.py

参数	原值	新值	说明
fetchWidth	8	3	取指宽度
decodeWidth	8	3	译码宽度
renameWidth	8	4	重命名宽度
dispatchWidth	8	4	分发宽度
wbWidth	8	3	写回宽度（仅兜底）
commitWidth	8	3	提交宽度
LQEntries	32	16	加载队列条目数
SQEntries	32	12	存储队列条目数
numPhysIntRegs	256	95	整数物理寄存器数
numPhysFloatRegs	256	48	标量 FP 物理寄存器数
numPhysVecRegs	256	48	向量物理寄存器数
numROBEntries	192	64	ROB 条目数

Phase 2：多 PRF + 分类写回端口

将单一寄存器文件拆分为独立 PRF，每个 PRF 有专属写回端口。

src/cpu/o3/BaseO3CPU.py

- 新增 3 个独立 PRF 写回端口宽度参数：
 - pregWBWidth = 3（整数 PRF）
 - vregWBWidth = 3（向量 PRF）
 - eregWBWidth = 2（标量 FP PRF）

src/cpu/o3/scoreboard.hh

- 将单一 `regScoreBoard` 拆分为 4 个独立 scoreboard：
 - `pregScoreBoard` — `IntRegClass`（整数）
 - `eregScoreBoard` — `FloatRegClass`（标量 FP）
 - `vregScoreBoard` — `VecRegClass`（向量）
 - `otherScoreBoard` — `VecPred` / `VecElem` / `Mat` / `CC`
- `getReg()`/`setReg()`/`unsetReg()` 按寄存器类别路由到对应 scoreboard

src/cpu/o3/scoreboard.cc

- 构造函数从 `(numPhysicalRegs)` 改为 `(numPregs, numEregs, numVregs, numOtherRegs)`

src/cpu/o3/iew.hh

- 新增 `pregWBWidth`/`vregWBWidth`/`eregWBWidth` 成员变量
- `wbWidth` 标记为 legacy fallback

Phase 3：8 条 Pipe + 执行延迟模型

为每个功能单元显式配置 `opLat`（操作延迟）和 `pipelined`（是否流水线化）。

src/cpu/o3/FuncUnitConfig.py

单元类	Pipe 映射	opClass	opLat	pipelined
IntALU	Pipe0/1: IU ALU	IntAlu	1	True
IntMultDiv	Pipe1: IU MLA / Pipe0/1: DIV	IntMult	3	True
		IntDiv	20	False
FP_ALU	Pipe6: VFPU ALU	FloatAdd/Cmp/Cvt/Bf16Cvt	3	True
FP_MultDiv	Pipe7: VFPU MA	FloatMult/FloatMultAcc	5	True
	Pipe6/7: FloatDiv/Sqrt	FloatDiv	12	False
		FloatSqrt	24	False
SIMD_Unit	Pipe6: VFPU ALU	SimdAdd/Alu/Cmp/etc	3	True
	Pipe7: VFPU MA	SimdMult/MultAcc	5	True
ReadPort	Pipe3: LSU Load	MemRead/etc	1	True
WritePort	Pipe5: LSU Store	MemWrite/etc	1	True
System_Unit	Pipe4: LSU Special	System	1	True

Phase 4：停顿/阻塞机制细化

从原有的 2-3 种停顿类型扩展为 9 种 C910 对齐的停顿原因。

src/cpu/o3/iew.hh

新增 `StallType` 枚举：

```
enum StallType {
    NO_STALL = 0,           // 无停顿
    ROB_FULL,              // ctrl_is_rob_full
    IQ_FULL,               // ctrl_is_iq_full
    VMB_FULL,              // ctrl_is_vmb_full
    TYPE_STALL,            // is_dis_type_stall (FENCE/barrier)
    DISPATCH_STALL,        // ctrl_is_dis_stall (ROB/IQ/VMB 任一满)
    IS_STALL,              // ctrl_is_stall (IS 级总停顿)
    DIV_STALL,             // iu_yy_xx_div_wb_stall
    VDIV_STALL,            // vfpu_idu_vdiv_wb_stall
    BACKEND_STALL,         // idu_hpcp_backend_stall
    NUM_STALL_TYPES
};
```

新增方法声明：`detectStallType()`, `isROBFull()`, `isIQFull()`, `isIQFullByType()`, `isVMBFull()`, `isTypeStall()`, `isDispatchStall()`

`src/cpu/o3/iew.cc`

- 实现 `detectStallType()` — 按优先级顺序检查各停顿条件
- 实现各细粒度检测方法（如 `isROBFull()` 检查 `freeROBEntries == 0`）
- `tick()` 中增加停顿周期计数：无停顿时累加 `no_stall`，否则按检测到的类型累加对应计数器
- 新增 `pipelineStallCycles` 统计（非零停顿周期）
- 新增 `iqEmptyCycles` 统计（无就绪指令周期）

Phase 5：分级刷新机制

将单一 squash 机制扩展为 4 种刷新类型，每种对应不同的刷新范围。

`src/cpu/o3/comm.hh`

```
enum class FlushType {
    FLUSH_FE = 0,          // 前端刷新 — 分支预测错误
    FLUSH_IS,              // 后端刷新 — 内存顺序违规
    FLUSH_GLOBAL,          // 全局刷新 — 异常/陷阱
    FLUSH_CSR,             // CSR 刷新 — 特权级/PTW 变更
    NUM_FLUSH_TYPES
};
```

- `TimeStruct::CommitComm` 新增 `flushType` 字段（默认 `FLUSH_GLOBAL`）
- 新增 `to_string(FlushType)` 辅助函数

Phase 6：前递路径细化

区分 EX1 组合逻辑前递（1 周期）和 EX2 时钟边沿写回（多周期）。

src/cpu/o3/inst_queue.hh

- IQStats 新增统计：
 - ex1ForwardInsts — EX1 单周期前递指令数
 - ex2WritebackInsts — EX2 多周期写回指令数
 - totalWakeDependents — 唤醒的依赖指令总数

src/cpu/o3/inst_queue.cc

- 在 scheduleReadyInsts() 中按 op_latency 分类计数：
 - op_latency == Cycles(1) → EX1 路径（直接进入 instsToExecute）
 - op_latency > Cycles(1) → EX2 路径（++wbOutstanding）
- 在 wakeDependents() 中累加 totalWakeDependents

Phase 7：性能计数器

补充 C910 HPCP（硬件性能计数器）事件。

src/cpu/o3/iew.hh — 新增统计

统计名	对应 HPCP	类型	说明
pipeIssueCount[8]	idu_hpcp_rf_pipe[0:7]_inst_vld	Vector	各 Pipe 发射计数
issueLatchFail[8]	idu_hpcp_rf_pipe[0:7]_lch_fail_vld	Vector	各 Pipe latch 失败
iqEmptyCycles	idu_had_iq_empty	Scalar	IQ 空周期
pipelineEmptyCycles	idu_had_pipeline_empty	Scalar	流水线空周期
pipelineStallCycles	idu_had_pipe_stall	Scalar	流水线停顿周期
stallCycles[9]	—	Vector	各停顿类型周期计数
backendStallCycles	idu_hpcp_backend_stall	Scalar	后端停顿周期
fenceSyncCount	—	Scalar	FENCE 同步事件

src/cpu/o3/iew.cc — opClass → Pipe 映射

在 executeInsts() 中，每条指令执行后根据其 opClass 映射到 Pipe 索引并累加 pipeIssueCount：

Pipe	索引	覆盖 opClass
Pipe0 (IU ALU)	0	IntAlu
Pipe1 (IU MLA/DIV)	1	IntMult, IntDiv
Pipe2 (BJU)	2	(预留, branch 不在 IEW 计数)

Pipe	索引	覆盖 opClass
Pipe3 (LSU Load)	3	MemRead, FloatMemRead, Simd*Load
Pipe4 (LSU Special)	4	System (FENCE/CSR)
Pipe5 (LSU Store)	5	MemWrite, FloatMemWrite, Simd*Store
Pipe6 (VFPU ALU)	6	FloatAdd/Cmp/Cvt, Simd*Alu, MatrixMov, crypto
Pipe7 (VFPU MA)	7	FloatMult/MultAcc, SimdMult/MultAcc, SimdDiv/Sqrt, Matrix

验证结果

使用 `hello world` 二进制在 O3CPU + caches 下运行通过，关键计数器输出：

```
ex1ForwardInsts          7  # 7 条单周期 ALU 指令
ex2WritebackInsts        0  # 无多周期指令
pipeIssueCount::pipe0_iu  7  # 全部走 Pipe0
stallCycles::no_stall     9  # 9 周期无停顿
stallCycles::rob_full     48 # 48 周期 ROB 满
pipelineStallCycles       48 # 48 周期流水线停顿
```